Cultra — Design Document

Cultra treats user input as a **goal**, not keyword search: parse intent → search mock Shenzhen venues → rank combinations → return multiple swipeable plans or a multi-stop itinerary. LLM (DeepSeek via OpenAI-compatible API; env vars `OPENAI_*`) handles JSON intent when configured; deterministic rules run search, rank, contingencies, and store writes.

1. Planning strategy

Intent parsing

Layer	When	Output
DeepSeek agent (`llm/agent.ts`)	API key in `.env.local`	`action` + structured `intent` (scenario, cuisines, party size, reserve time, quiet dining, delivery add-ons, full-day patterns, invites)
Rules fallback (`parse_intent.ts`)	No key or LLM error	Same `ParsedIntent` via regex/keywords

Both paths post-process: `finalizeOutingPlanIntent`, `syncReserveTimeFromUserText` (user clock beats LLM), `inferTimeOfDayFromClock`, delivery-kind extraction.

Key signals: Budget from last five same-scenario purchases (±40%/person); price tiers override. Quick-lunch → 2 km, 15 min prep. Party size from explicit counts or LLM. **Quiet dining** filters `reservationLoad > 55%` and boosts low-load venues. **Friends scenario** uses `fetch_friend_history()` — purchase history only for mutual friends; favorites/check-ins always apply.

Plan generation

Mode	Flow
Simple combos	Filtered activity + restaurant pools → ≤3 unique pairs → `check_queue_status` → `rank_plans` → delivery add-ons → one-stop preview
Full itinerary (`wantsFullItinerary`)	`build_itinerary_plans()` — 2–3 stops, travel steps, cost summary, `splitBillEligible` when ≥2 paid stops

Ranking (`rank_plans.ts`): 30% distance · 55% cuisine/diet/price/friend history/quiet load · 15% queue wait (20+ min penalized).

Search widen (empty pools): no activities → afternoon @ 15 km, full-day @ 20 km; no restaurants → drop cuisine/diet/quiet, 15–20 km, rating ≥ 3.5. Strict district with empty pool → no widen; prompt user to change district.

Dynamic Contextual Prompt Chips allow users to trigger AI plan generations with a single tap.

2. Tool call chain

Planner — `POST /api/plans` → `handle_chat.ts`

```
messages + ChatContext
  → analyzeWithLlm | analyzeWithRules
  → ParsedIntent + action (converse | cuisine_info | show_plans | execute |
invite_friends)
  → post-process intent
  → [early] delivery add-on → applyDeliveryAddonsFromMessage
  → [early] contingency → plan_contingency
  → [early] invite → invite_friends → ActivityRoom
  → branch: converse | cuisine_info | execute (acceptPlan) | show_plans →
generate_plans()
```

`generate_plans()`: `fetch_friend_history`[friends] → `search_activities` → `search_restaurants` → [widen if empty] → `build_itinerary_plans` | `buildUniqueCombos` → `check_queue_status` → `rank_plans` → `attachDeliveryAddonsToPlan` → `attachOneStop` (preview).

Acceptance — `POST /api/execute`

```
swipe/confirm → acceptPlan (order | reserve | share_only) → demoStore
  → send_plan_message → execute_one_stop (reservation / traffic / reminder)
```

Order support — `POST /api/order` (chat) → `replyToOrderSupport`

```
tryApplyDeliveryAddonsToOrder → tryApplyContingency → replyWithLlm |
getOrderChatReplyRules
```

Shortcuts: `GET /api/nearby` — tap restaurant → `plansFromRestaurantIds` without full chat. Profile nation drives nearby nationality column.

```
flowchart LR
  U[User] --> HC[handle_chat]
  HC --> LLM{DeepSeek?}
  LLM -->|yes| A[JSON agent]
  LLM -->|no| R[Rules]
  A --> GP[generate_plans]
  R --> GP
  GP --> RK[rank_plans] --> UI[SwipePlanDeck]
  UI --> EX[execute] --> ST[demoStore] --> SC[support_chat]
```

3. Exception handling

Case	Mechanism
Rain / bad weather	`weather_activity` → swap outdoor for indoor, same district
Crowd / long queue / sold out	`restaurant_issue` → same-cuisine nearby alternative
Explicit swap	`swap_restaurant` / `swap_activity` → backup search in plan district
No backup	User message; suggest different district or cuisine
No venues match	Widen search (§1); zero plans → cuisine list or district hint
Unknown / non-mutual friend	Empty purchase history; favorites still used
No API key / LLM unreachable	Rules-only; planner shows “basic mode” notice
LLM timeout / error	Silent fallback to rules (planner + support chat)
Delivery add-on, no restaurant	Prompt to pick a plan with a dining stop
One-stop without reserve	Runs only blocks in `intent.oneStop` (traffic/reminder optional)
No active order	Orders page redirects to planner

Contingencies run in **planner chat** (existing plan context) and **order support chat** (`syncOrderFromPlan`). Queue times, traffic ETAs, reservations, and rider positions are **mocked**. State: `demoStore` (`globalThis._cultraStore`), in-memory for the server session.

4. List of features

One-Stop Agent	1-click execution for complex itineraries, automated remote queueing, integrated on-demand delivery (外卖/跑腿) for event add-ons (eg gifts, champagne, etc).
Planner	Scenario-aware engine generating multiple plan alternatives dynamically scaled by duration and group type. Features cultural filtering, predictive budgeting (if not specified by user, it takes the average price of last 5 past purchases), quiet-dining optimization(recommends restaurants with lesser reservation load or queue), logistical extras (traffic ETAs/reminders), and real-time AI chat for instant backup plans (e.g., weather complications).
Friends page	Syncs friends via ID, generates shared Party Rooms based on wishlist overlaps, and drives engagement with "Friends Also Want" mutual recommendations on activities or restaurants.
Order, Data, Profile Pages	Seamless group bill-splitting, live waitlist tracking, local hotspot discovery, and deep cultural personalization.